SCHOOL MANAGEMENT SYSTEM
Complete MySQL Project
==========

1. CREATE DATABASE
```
CREATE DATABASE school_management_system;
```

2. SELECT DATABASE
```
USE school_management_system;
```

==========
3. CREATE STUDENTS TABLE
==========

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50)
);
```

==========
4. CREATE SUBJECTS TABLE
==========

```
CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(100)
);
```

==========
5. CREATE EXAMS TABLE
==========

```
CREATE TABLE Exams (
    exam_id INT PRIMARY KEY,
    exam_name VARCHAR(100),
    exam_date DATE
);
```

```
 ================================================
 6. CREATE MARKS TABLE
 ================================================

CREATE TABLE Marks (
    mark_id INT PRIMARY KEY,
    student_id INT,
    subject_id INT,
    exam_id INT,
    score INT,

    FOREIGN KEY (student_id)
        REFERENCES Students(student_id),

    FOREIGN KEY (subject_id)
        REFERENCES Subjects(subject_id),

    FOREIGN KEY (exam_id)
        REFERENCES Exams(exam_id)
);


 ================================================
 7. INSERT STUDENT DATA
 ================================================

INSERT INTO Students
(student_id, first_name, last_name)
VALUES
(1, 'Yadagiri', 'Goud'),
(2, 'Narsimha', 'Roa'),
(3, 'Mallaiah', 'Yadav'),
(4, 'Padma', 'Patil'),
(5, 'Laxman', 'Konda'),
(6, 'Vikram', 'Singh'),
(7, 'Ramesh', 'Kumar'),
(8, 'Sunitha', 'Reddy'),
(9, 'Kiran', 'Chary'),
(10, 'Chandraiah', 'Varma');


 ================================================
```

8. INSERT SUBJECT DATA
==========

```
INSERT INTO Subjects
(subject_id, subject_name)
VALUES
(1, 'Mathematics'),
(2, 'Science'),
(3, 'English'),
(4, 'Hindi'),
(5, 'Social Studies');
```

==========
9. INSERT EXAM DATA
==========

```
INSERT INTO Exams
(exam_id, exam_name, exam_date)
VALUES
(1, 'Half-Yearly', '2024-09-15'),
(2, 'Annual', '2024-03-20');
```

==========
10. INSERT MARKS DATA
==========

```
INSERT INTO Marks
(mark_id, student_id, subject_id, exam_id, score)
VALUES
(1, 1, 1, 1, 95),
(2, 1, 2, 1, 62),
(3, 1, 3, 1, 56),
(4, 1, 4, 1, 72),
(5, 1, 5, 1, 70),

(6, 2, 1, 1, 69),
(7, 2, 2, 1, 63),
(8, 2, 3, 1, 61),
(9, 2, 4, 1, 98),
(10, 2, 5, 1, 89),

(11, 3, 1, 1, 60),
```

```
(12, 3, 2, 1, 92),
(13, 3, 3, 1, 82),
(14, 3, 4, 1, 57),
(15, 3, 5, 1, 56),

(16, 4, 1, 1, 60),
(17, 4, 2, 1, 68),
(18, 4, 3, 1, 69),
(19, 4, 4, 1, 87),
(20, 4, 5, 1, 93),

(21, 5, 1, 1, 56),
(22, 5, 2, 1, 90),
(23, 5, 3, 1, 67),
(24, 5, 4, 1, 96),
(25, 5, 5, 1, 89),

(26, 6, 1, 1, 81),
(27, 6, 2, 1, 69),
(28, 6, 3, 1, 83),
(29, 6, 4, 1, 92),
(30, 6, 5, 1, 72),

(31, 7, 1, 1, 55);


 ================================================
 11. VERIFY INSERTED DATA
 ================================================

SELECT * FROM Students;

SELECT * FROM Subjects;

SELECT * FROM Exams;

SELECT * FROM Marks;


 ================================================
 12. QUERY 1: RETRIEVE ALL STUDENTS
 ================================================

SELECT first_name, last_name
```

```
FROM Students;


 ================================================
 13. QUERY 2: FILTER SCORES GREATER THAN 80
 ================================================

SELECT *
FROM Marks
WHERE score > 80;


 ================================================
 14. QUERY 3: SORT EXAMS BY DATE
 ================================================

SELECT *
FROM Exams
ORDER BY exam_date DESC;


 ================================================
 15. QUERY 4: STUDENT MARKS DETAILS
 ================================================

SELECT
    st.first_name,
    st.last_name,
    su.subject_name,
    m.score
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
JOIN Subjects AS su
    ON m.subject_id = su.subject_id;


 ================================================
 16. QUERY 5: SUBJECT AVERAGES
 ================================================

SELECT
    su.subject_name,
    ROUND(AVG(m.score), 2) AS average_score
```

```
FROM Marks AS m
JOIN Subjects AS su
    ON m.subject_id = su.subject_id
GROUP BY su.subject_id, su.subject_name;


 ================================================
 17. QUERY 6: COUNT MARKS FOR A STUDENT
 ================================================


SELECT COUNT(m.mark_id) AS total_marks_records
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
WHERE st.first_name = 'Aarav'
  AND st.last_name = 'Sharma';



SELECT COUNT(m.mark_id) AS total_marks_records
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
WHERE st.first_name = 'Yadagiri'
  AND st.last_name = 'Goud';


 ================================================
 18. QUERY 7: TOP PERFORMER
 ================================================

SELECT
    st.first_name,
    st.last_name,
    m.score
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
ORDER BY m.score DESC
LIMIT 1;


 ================================================
```

19. QUERY 8: PASS / FAIL LABE
==========

```
SELECT
    st.first_name,
    st.last_name,
    su.subject_name,
    m.score,
    CASE
        WHEN m.score >= 70 THEN 'Pass'
        ELSE 'Fail'
    END AS status
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
JOIN Subjects AS su
    ON m.subject_id = su.subject_id;
```

==========

20. QUERY 9: TOTAL SCORE PER STUDENT

==========

```
SELECT
    st.student_id,
    st.first_name,
    st.last_name,
    SUM(m.score) AS total_score
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
GROUP BY
    st.student_id,
    st.first_name,
    st.last_name;
```

==========

21. QUERY 10: SUBJECTS WITH AVERAGE SCORE < 75

==========

```
SELECT
    su.subject_name,
    AVG(m.score) AS avg_score
```

```
FROM Marks AS m
JOIN Subjects AS su
   ON m.subject_id = su.subject_id
GROUP BY su.subject_id, su.subject_name
HAVING AVG(m.score) < 75;


 ================================================
 22.  COUNT TOTAL STUDENTS
 ================================================

SELECT COUNT(*) AS total_students
FROM Students;


 ================================================
 23. COUNT TOTAL SUBJECTS
================================================

SELECT COUNT(*) AS total_subjects
FROM Subjects;


-- ================================================
24. FIND SCORES OF 90 OR ABOVE
-- ================================================

SELECT
   st.first_name,
   st.last_name,
   su.subject_name,
   m.score
FROM Marks AS m
JOIN Students AS st
   ON m.student_id = st.student_id
JOIN Subjects AS su
   ON m.subject_id = su.subject_id
WHERE m.score >= 90
ORDER BY m.score DESC;


 ================================================
 25.FIND SCORES BELOW 35
 ================================================
```

```
SELECT
    st.first_name,
    st.last_name,
    su.subject_name,
    m.score
FROM Marks AS m
JOIN Students AS st
    ON m.student_id = st.student_id
JOIN Subjects AS su
    ON m.subject_id = su.subject_id
WHERE m.score < 35;


 ================================================
 END OF SCHOOL MANAGEMENT SYSTEM
================================================
```